Lecture plan · for the lecturer

Assessing programming at scale

Programming · University · Lecture 4

Time
60 min

Materials
Projector or board; The lecture slides and the seminar sheet from the kit; Phones or laptops with Darsi, for the lab

Learning goals

- Design strong test suites.
- Give useful feedback.
- Protect integrity by design.

Activity: Break the submission

1. Warm-up (5 min): Show the buggy median and ask students whether the visible test catches it.
2. Test (20 min): Pairs write hidden tests that break three passing submissions.
3. Feedback (15 min): Write a hint for each failing case that does not give the fix.
4. Policy (15 min): Draft a one-paragraph policy and one live-modification task.
5. Wrap-up (5 min): Students write the muddiest point of the lecture and one question; read them before next week.

Discussion

- Which test broke the most submissions?
- When does a hint become a fix?
- What will your policy allow?

Exit ticket

- Write one hidden test for your next assignment.

Try it in your next lecture
Show a submission that passes the visible tests and ask pairs to write one hidden test that breaks it; keep the best tests for next term's suite.

Seminar sheet · for students

Assessing programming at scale

Programming · University · Lecture 4

Name _____ Group _____ Date _____

1 **Which change best shows a student understands their submitted code?**

◯ An AI-detection tool ◯ More visible tests ◯ Modify it live and explain

2 **Fix the median. This function passes the visible test, an odd-length list. Run it against the hidden tests, read what they report, then fix it.**

```
def median(xs):
    s = sorted(xs)
    n = len(s)
    return s[n // 2]
```

Write your program:

3 **Which strengthen an autograded assignment? Choose all that apply.**

☐ Checking broken reference solutions fail ☐ Hidden edge-case tests

☐ Showing the full expected output of every hidden test

☐ A short code-reading or oral component

4 **A high similarity score from MOSS is sufficient proof of misconduct.**

◯ True ◯ False

5 **Match each problem to a design response.**

Code hard-codes visible answers	• • • •	Submission limits
Endless trial-and-error resubmits	• • • •	Hidden tests
AI-written code not understood	• • • •	Mutation testing
Weak test suite	• • • •	Live modification and explanation

Answers · for the lecturer

Assessing programming at scale

Programming · University · Lecture 4

1 Modify it live and explain

Live modification shows understanding directly.

2

```
def median(xs):
    s = sorted(xs)
    n = len(s)
    if n % 2 == 1:
        return s[n // 2]
    return (s[n // 2 - 1] + s[n // 2]) / 2
```

With an even number of items the median is the mean of the two middle ones, s[n // 2 - 1] and s[n // 2].

3 Hidden edge-case tests; Checking broken reference solutions fail; A short code-reading or oral component

Revealing hidden outputs lets students code to the tests.

4 False

Similarity can arise from starter code or common solutions; it justifies a conversation, not a verdict.

5

Code hard-codes visible answers — Hidden tests
Endless trial-and-error resubmits — Submission limits
AI-written code not understood — Live modification and explanation
Weak test suite — Mutation testing

Each response targets one failure of the assessment.

For you to think about

What is your current policy on AI assistance in programming assignments, and is it written where students see it?